Dependency Management and Static Stability

keep working without changing anything

Watch the 4-min video

In one line: a statically stable system keeps serving when a dependency or the control plane fails, without having to change anything, because capacity is pre-provisioned and config, credentials and routes are cached as last known good; keep hard dependencies few and prove soft ones are soft.

1 Same outage, two outcomes

A cloud control plane has an outage: nobody can launch instances or change settings. One company's services keep running. Another's crash as soon as they restart, because they can't fetch their configuration at startup. The difference is **static stability**: the ability to keep working when dependencies fail, **without needing to react, scale or reconfigure**. For every dependency ask: if this goes down, do we keep serving?

2 Control plane vs data plane

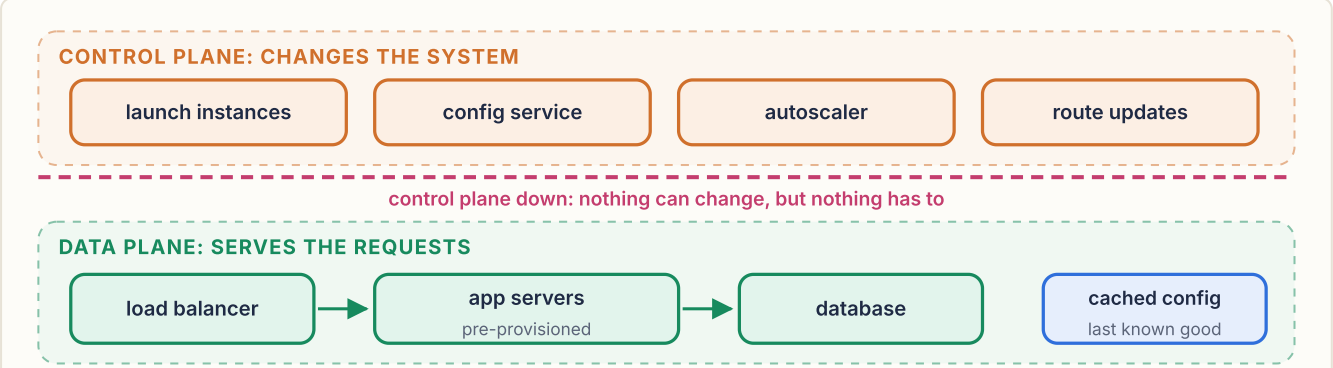


Figure 1. The data plane must not need the control plane to keep serving.

The **data plane** does the real work: serving requests, moving packets, reading data. The **control plane** changes the system: launching instances, updating config, changing routes. Control planes are more complex and change more often, so they fail more often. Design the data plane to keep running, with what it already has, while the control plane is down. AWS's Builders' Library describes this for EC2 and Availability Zones.

3 How to build it

- **Pre-provision capacity:** run enough in each zone that losing one needs no new launches mid-crisis (lesson 10's zone sizing). Launching is a control-plane action.
- **Cache locally** config, credentials, feature flags and service lists; keep using the **last known good** copy if refresh fails.
- **Push, don't pull per request:** distribute config to services ahead of time instead of fetching it on the request path.
- **Start without hard dependencies:** boot from cached values, report "degraded", refresh in the background.

Startup is the common trap. If a service refuses to start unless the config service, the flag service and the secrets store all answer, any one of their outages blocks every restart, deploy and autoscaling event, exactly when you need them.

4 Hard vs soft dependencies

Dependency	Hard or soft?	If it fails
main database (writes)	hard	writes fail; keep it highly available (lessons 10–11)
config / feature-flag service	soft	use cached last known good values
recommendations, analytics	soft	fallback or skip (lesson 8)
secrets store	soft at runtime	keep cached credentials until they expire; renew early
autoscaler / instance launch	soft if pre-provisioned	serve with existing capacity

Keep hard dependencies **few**, write them down, and **test** that the soft ones really are soft by switching them off, which is a chaos experiment (lesson 17).

WORKED EXAMPLE: CONFIG CACHED ON DISK

Every service fetches its configuration every **30 s** and writes it to a local `config.json`. On startup it reads the file first, so it can start immediately. One day the config service is down for an **hour**. Running services keep their last known good config, new pods start from the cached file, and autoscaling still works. Users see nothing; config *changes* simply wait until the service recovers.

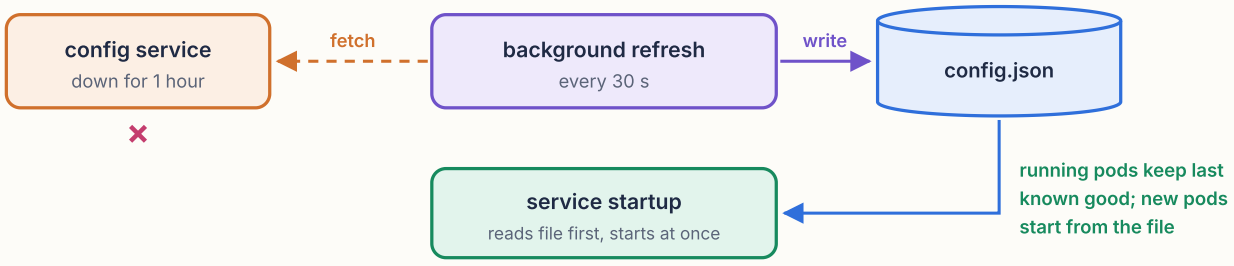


Figure 2. The config service is on the change path, not the serving path, so its outage only delays updates.

COMMON MISTAKES

- Services that must reach config, flags or secrets to boot.
- Relying on autoscaling or new launches to survive a zone loss.
- Fetching config or permissions on every request instead of caching.
- "Soft" dependencies that were never tested and turn out to be hard.

5 Key terms

Static stability: keeps working through failures without changing anything

Data plane: the part that serves requests

Control plane: the part that changes and manages the system

Last known good: most recent valid cached state

Hard dependency: can't serve without it

Soft dependency: can fail with a fallback

6 Interview questions

Q1. What is static stability? BASIC

A system's ability to keep working when a dependency or control plane fails, without making any changes, relying on pre-provisioned capacity and cached, last-known-good state.

Q2. What's the difference between the control plane and the data plane? BASIC

The data plane serves the actual requests; the control plane changes and manages the system. The data plane shouldn't need the control plane to keep running.

Q3. Your services can't start when the config service is down. How do you fix it? SCENARIO

Cache config to local disk on every refresh, start from the cached file, report degraded and refresh in the background.

Q4. How do you survive a zone loss if instance launches are failing? SCENARIO

Pre-provision: run enough capacity in each zone that the remaining zones carry full load without launching anything.

Q5. Why are control planes less reliable than data planes? TRICKY

They are more complex, change more often and do more coordination, so they fail more often; a data plane mostly repeats simple work.

Q6. Is caching last known good config always safe? TRICKY

Mostly, but it can keep serving a bad config or revoked credentials; add validation before accepting new config and a maximum age for sensitive values.

Next: SD-L-001 · Monolith vs Microservices backendroadmap.com